SPIN - Seminar

RAFT

Formal Methods for AI-Based Systems Engineering (FMAISE)

William Tossici (1844504)
Academic Year 2025/2026

Reference: D. Ongaro, J. Ousterhout, *In Search of an Understandable Consensus Algorithm*, USENIX ATC, 2014.

SAPIENZA
UNIVERSITÀ DI ROMA

Table of Contents

The case study: formally verifying RAFT in SPIN

1 Introduction

- RAFT is a consensus algorithm for replicated state machines: it elects a leader and replicates a log across the nodes.
- Distributed protocols are hard to get right: subtle bugs hide in message interleavings, timeouts and crashes, and testing rarely reproduces them.
- **Goal:** model the core of RAFT in Promela and verify its key properties *automatically and exhaustively* with SPIN.

What SPIN gives us:

- Systems as concurrent processes communicating over channels (Promela).
- Properties written in LTL; SPIN explores **every** reachable state.
- We check *safety* (at most one leader per term, log matching) and *liveness* (a leader is eventually elected).

Modeling overview: one base model, four extensions

- Each of the N nodes is a Promela process (`proctype Node`), talking through one bounded, lossy inbox channel per node.
- Every node runs the same state machine: `FOLLOWER` / `CANDIDATE` / `LEADER`.
- Verification relies on global "ghost" variables (e.g. `leader_count`) and LTL claims: they observe the protocol without changing it.

Structure of the work: a base model (leader election), then four orthogonal extensions that each add one idea:

- explicit time
- crash / restart
- log replication

Table of Contents

2 Translating Raft to Promela

From RAFT to Promela: why simplify

- SPIN verifies by exploring **all** reachable states $\Rightarrow$ combinatorial explosion.
- Every abstraction below keeps the state space **finite and small**, while staying *sound* for safety.

Anti-explosion toolkit (recurs in every model):

- Single-slot channels: `BUF_SIZE = 1`
- Finite caps: `MAX_TERM`, `MAX_HB`, `MAX_TIME`
- Group sends inside `atomic{}` (no intermediate interleavings)
- Canonicalization of "dead" variables (`clear_msg`)

Table of Contents

3 Base model

Base — a node is a state machine

3 Base model

```
1  proctype Node(byte id) {
2    byte state = FOLLOWER, currentTerm = 0, votedFor = 255,
3         votesRcv = 0, hb_count = 0;
4    do
5    :: (state == FOLLOWER)  -> ...  /* receive msg | election timeout | idle
         */
6    :: (state == CANDIDATE) -> ...  /* count votes | step-down | idle
         */
7    :: (state == LEADER)    -> ...  /* heartbeat | step-down | idle
         */
8    od
9  }
```

- Three roles, a single infinite `do`: this is RAFT's state machine.
- Nondeterminism among the branches models "what can happen now".
- Finiteness guaranteed: every variable has a bounded domain.

Base — the network: *lossy* channels


```
1  inline net_send(dest, mtyp, frm, trm, lp) {
2    if
3    :: nfull(inbox[dest]) -> inbox[dest] ! mtyp, frm, trm, lp   /* delivered */
4    :: skip                                                     /* packet lost */
5    fi
6  }
```

- Every send has **two nondeterministic outcomes**: delivered *or* lost ⇒ safety must hold under any message loss.
- Single-slot `inbox`: if full, the message is dropped (never blocking).
- `nfull` (not `!full`): Promela forbids negating channel predicates.

Base — timeout *without a clock* + atomic broadcast


```
1  :: (currentTerm < MAX_TERM) ->              /* election timeout */
2     currentTerm++; votedFor = id; votesRcv = 1;    /* self-vote */
3     state = CANDIDATE;
4     atomic {                                 /* RequestVote to all, in one block
          */
5       i = 0;
6       do
7       :: (i < N) -> if :: (i != id) ->
8                           net_send(i, RequestVote, id, currentTerm, LOG_PRI(id
                                ))
9                     :: else -> skip fi; i++
10      :: (i >= N) -> break
11      od
12    }
```

- In the base, the timeout is **pure nondeterminism**: no time, "fires whenever it can".
- `atomic`: the broadcast is a single transition ⇒ cuts useless interleavings.
- `LOG_PRI(id)=id+1`: the *Leader Restriction* abstracted as static priorities.

Base — winning, safety monitor and the LTL property

3 Base model

```
:: (msg_type == VoteGranted) && (msg_term == currentTerm) ->
   votesRcv++;
   if :: (votesRcv > N/2) ->              /* majority */
         state = LEADER; hb_count = 0;
         leader_count[currentTerm]++       /* <-- monitor for verification */
      :: else -> skip
   fi

ltl safety_one_leader {
  [] ( leader_count[0] <= 1 && leader_count[1] <= 1
    && leader_count[2] <= 1 && leader_count[3] <= 1 )
}
```

- `leader_count[t]` is not part of the protocol: it is a *monitor* for the LTL.
- Property: in every state, **at most one leader per term** (RAFT §5.2).

Table of Contents

4 Extension: explicit time

Timed — the timeout becomes a *deadline*

4 Extension: explicit time

```
1  inline arm_election_timer(id) {        /* duration chosen NON-
       deterministically */
2    if :: elect_deadline[id] = now + T_SHORT
3       :: elect_deadline[id] = now + T_LONG
4    fi
5  }
6
7  /* the timeout is now GUARDED by the clock: */
8  :: (now >= elect_deadline[id]) && (currentTerm < MAX_TERM) -> ...
```

- Nondeterminism shifts from "who acts" to "how long the timeout lasts".
- Asymmetric durations (`T_SHORT`/`T_LONG`): SPIN explores "who expires first".
- A live leader, by sending heartbeats, *re-arms* the followers' timers: it stabilizes.

Timed — advancing time: *variable time advance*

4 Extension: explicit time

```
1  proctype Advance() {
2    do
3    :: atomic {
4         all_quiescent && (now < MAX_TIME) ->    /* only if nobody can act now
                 */
5         mind = MAX_TIME;                        /* find the next deadline
                    */
6         k = 0;
7         do :: (k < N) -> ... /* min of elect_deadline[k], hb_deadline[k] */ ;
                k++
8            :: (k >= N) -> break od;
9         now = mind                              /* JUMP to the next event
                    */
10      }
11   :: else -> break
12   od
13 }
```

- Time **does not advance by +1**: it jumps straight to the next event instant.
- Avoids filling the space with "empty" states where nothing happens.
- Fires only when the system is quiescent (maximal progress: instantaneous actions first).

Table of Contents

Crashes — crash + *persistence* (stable storage)

5 Extension: crash / restart

```
1 inline do_crash(decr_lc) {
2   atomic {
3     if :: (decr_lc == 1) -> leader_count[currentTerm]-- :: else -> skip fi;
4     crash_count++; state = CRASHED;
5     /* VOLATILE: reset */              votesRcv = 0; hb_count = 0; clear_msg()
          ;
6     /* PERSISTENT: currentTerm and votedFor untouched (survive the crash) */
7   }
8 }
```

- `votedFor` **must** persist: otherwise a restarted node would vote twice in the same term $\Rightarrow$ two leaders (safety violated).
- The crash is offered **only at idle points** (quiescent): no state multiplication at every active step.
- `MAX_CRASHES` budget: keeps the space finite.

Table of Contents

6 Fusion: time + crash

Timed+crashes fusion — the *timed* restart


```
1  inline do_crash(decr_lc) {
2    atomic {
3      ... state = CRASHED; down[id] = 1;
4      /* persistence: currentTerm, votedFor unchanged */
5      elect_deadline[id]    = OFF;          /* a dead node has no events of its
           own... */
6      hb_deadline[id]       = OFF;
7      restart_deadline[id] = now + T_DOWN    /* ...except its scheduled wake-up
            */
8    }
9  }
```

- The leader crashes ⇒ heartbeats stop ⇒ followers time out *during the downtime* ⇒ failover: a scenario controllable in time.
- Dead node's timers set to `OFF`: otherwise `Advance` would never advance time again.

Table of Contents

Log — the *log entry* abstraction

7 Extension: log replication

```
byte logv[N * MAX_LOG];                    /* global, flattened log */
#define LOG(n, k)  logv[(n)*MAX_LOG + (k)]    /* entry = just its TERM, 0 = empty */

inline last_log(nid, lli, llt) {           /* node's (lastLogIndex, lastLogTerm) */
  if :: (LOG(nid,1) != 0) -> lli = 2; llt = LOG(nid,1)
     :: (LOG(nid,1) == 0) && (LOG(nid,0) != 0) -> lli = 1; llt = LOG(nid,0)
     :: else -> lli = 0; llt = 0
  fi
}
```

- An entry is **just the term** it was created in: the payload is irrelevant for safety.
- Only `MAX_LOG=2` slots per node: enough for the consistency check.
- `last_log` implements the *real Leader Restriction* (§5.4.1), no longer `LOG_PRI`.

Log — consistency check and *Log Matching*

7 Extension: log replication

```
1  :: (msg_p1 == 1) ->                          /* the leader replicates the entry at
       slot 1 */
2     if :: (LOG(id,0) != 0) && (LOG(id,0) == msg_p2) ->    /* prevLogTerm
           matches? */
3           LOG(id,1) = msg_term;                          /* accept */
4           net_send(msg_from, AppendOk,   id, currentTerm, 1, 0, 0)
5        :: else ->
6           net_send(msg_from, AppendFail, id, currentTerm, 0, 0, 0)  /* reject
                */
7     fi
8
9  ltl log_matching {
10   [] ( (LOG(0,1) != 0 && LOG(0,1) == LOG(1,1)) -> (LOG(0,0) == LOG(1,0)) &&
         ... )
11 }
```

- Accept entry k **only if** entry $k-1$ matches the leader's.
- `AppendEntries` content chosen nondeterministically: no `nextIndex[]`.
- Property: same entries at the same index $\Rightarrow$ identical prefixes (§5.3).

Summary

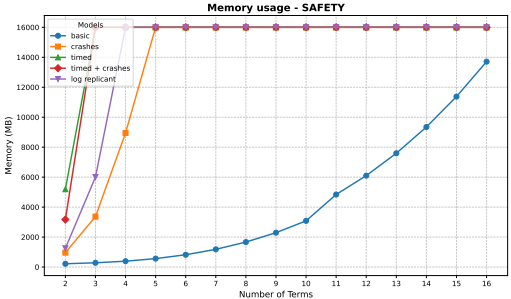
7 Extension: log replication

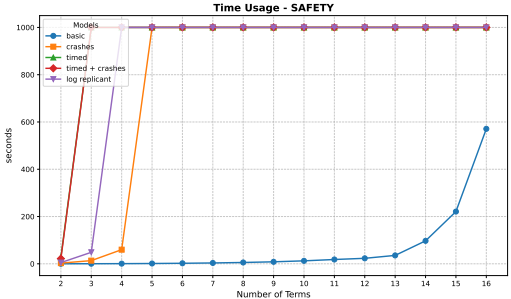
- A **common core** (state machine + lossy network + monitor + LTL) and four orthogonal extensions: time, crash, log.
- Each extension adds *one* idea, reusing the same skeleton.
- Every simplification either restricts behaviors in a *sound* way for safety, or affects only liveness.

Table of Contents

Results: memory and time - SAFETY

8 Results

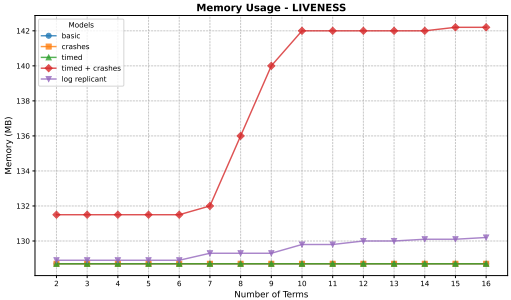


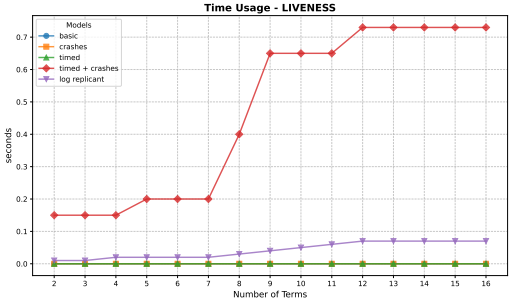


Runs on a MacBook Pro (Apple M5, 16 GB RAM).

Results: memory and time - LIVENESS

8 Results





Runs on a MacBook Pro (Apple M5, 16 GB RAM).